

university of groningen

Performance Engineering of Direction-Optimising BFS (GAPBS)

WMCS045-05 · Group 1

Sergiu Alexandru Macra **Cosmin Andrei Stamate**

Bernoulli Institute

2026-10-20

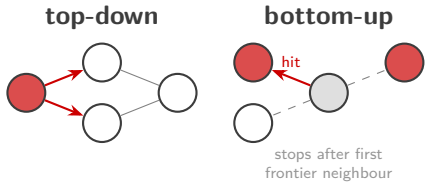
Overview



1. Application and systems
2. Measurement method and tool validation
3. Performance analysis
4. Performance model
5. Optimisations
6. Conclusions

Application: direction-optimising BFS

- GAP Benchmark Suite, `bfs`: CSR graph, OpenMP
- **Top-down** (`TDStep`): frontier pushes to neighbours, random CAS on `parent[v]`
- **Bottom-up** (`BUStep`): every unvisited vertex searches its neighbours for a frontier vertex and *stops at the first hit*
- Switch by $\alpha = 15$, $\beta = 18$; conversions `QueueToBitmap`, `BitmapToQueue`
- Input `kron20`: 2^{20} vertices, 15.7 M edges, 136 MB



Edge counters we added: a search examines only **4.7 %** of the 31.4 M directed edges. TEPS overstates the real work by $\approx 20\times$.

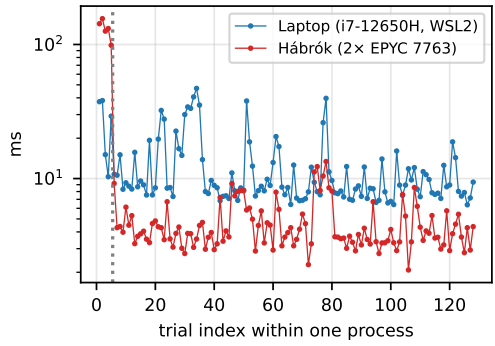
Systems

	Laptop	**Hábrók node**
CPU	Intel i7-12650H (6P+4E), WSL2: 10 vCPUs	2× AMD EPYC 7763, 128 cores, 2 NUMA
STREAM triad (all / 1 thread)	15.9 GB/s / 12.3 GB/s	327 GB/s / 34.7 GB/s
DRAM latency (1 GB)	376 ns	99 ns
Measured MLP per core	15.7	21.7
Counters	none (WSL2 hides the PMU)	PAPI 7.1 (built by us), pecount

- No HAICGU access ⇒ Hábrók instead. No perf, PAPI or numactl there: we **built PAPI** and wrote **pecount** (`perf_event_open`, 200 lines)
- Everything runs from one batch script per platform; 30 variants via compile-time switches

Measurement method

- 1 discarded run, then 5 processes $\times$ 24 searches
- **Drop trials 1–5** of each process: warm-up is up to 50$\times$ slower on Hábrók
- Report **median and IQR**; GTEPS $= m/t$
- Significant only if the difference exceeds **both IQRs**
- **Every variant must pass the BFS verifier** before it is timed



Do the tools measure what we think?

Check	Result	Verdict
3 wall-clock sources (laptop)	0.679 s, 0.717 s and 0.69 s	agree
perf task-clock vs. `time`	6.00 s vs. 6.05 s	agree
pecount vs. perf	page faults 36,422 vs. 36,423	agree
PAPI regions vs. pecount	misses 99.1 %, cycles 95 %	agree
Known answer: ALU chain	**5.000** instr., **4.000** cycles / iter.	= `imul`+`add` loop
Known answer: 1 GB chase	**1.0007** misses / step	exact
gprof	100 % in `frame_dummy`	unusable for OpenMP
pecount, multiplexed	2.4 misses / step	scaling breaks

Verification found a hidden data race

- `BUStep` sets bits in a shared bitmap **without atomics**
- Safe only if each thread's chunk starts on a 64-bit word
- Upstream `dynamic,1024` satisfies this *by accident*
- $\Rightarrow$ schedule sweep restricted to chunk sizes that are multiples of 64

`OMP_SCHEDULE`	Laptop	Hábrók
`dynamic,1024`	8/8	8/8
`dynamic,64`	8/8	8/8
`static,1000`	4/8	0/8
`dynamic,100`	0/8	0/8
`guided`	0/8	0/8

correct BFS trees out of 8

Where does the time go?



Laptop, perf sampling (10 threads)

OpenMP runtime (barrier spin)	50.0 %
`BUStep`	35.8 %
`InitParent`	5.7 %
`TDStep`	3.1 %
conversions + rest	5.4 %

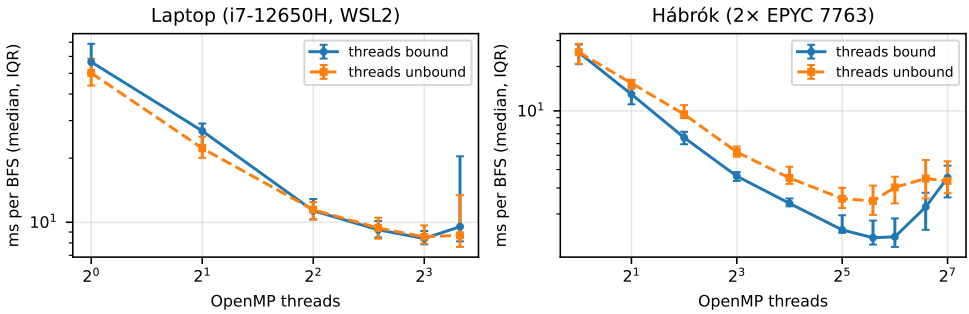
39 % of `BUStep` samples on the miss to the start of a neighbour list

Hábrók, PAPI per kernel (1 thread)

Kernel	Cycles	IPC	Misses
`BUStep`	78 %	0.77	0.32 / vertex
`InitParent`	14 %	0.89	–
`TDStep`	4 %	1.01	0.75 / edge
conversions	4 %	1.6–4.8	–

memory-bound: low IPC; TLB irrelevant with THP (4.2k misses / search)

Thread scaling



- Laptop: flat beyond 8 threads. Hábrók: best at **48–64 bound threads** (one socket), slower again at 128
- The server has 13× the cores and 20× the bandwidth, but is only 3× faster. Why?

Model: bytes and random accesses, not FLOPs



- Roofline in FLOP/byte: BFS has no FLOPs
- ECM: needs predictable steady-state loops
- $\Rightarrow$ **information-exchange model** from measured work

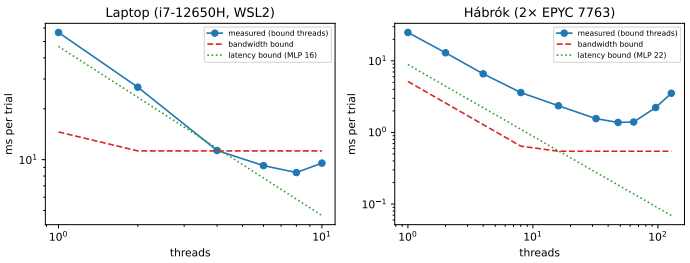
$$T_{\text{bw}}(p) = \frac{Q}{\min(pB_1, B_{\text{all}})}$$

$$T_{\text{lat}}(p) = \frac{N_{\text{rand}}\, L_{\text{DRAM}}}{\text{MLP} \cdot p}$$

All parameters measured: edge counters, STREAM, pointer chase, MLP benchmark

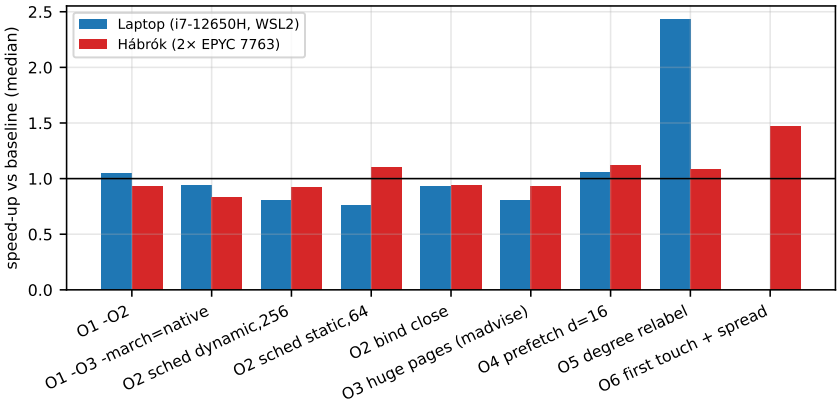
Traffic per `kron20` search	MB
`InitParent`	12.6
bottom-up sweeps	31.8
bottom-up neighbour lines	**106.2**
top-down edges + `parent[v]`	19.5
conversions, final pass	8.7
Q	**178.8**
N_{rand}	1.95 M

Model vs. measurement: three regimes



1. **1 thread: latency-bound**
   laptop within 21 %; Hábrók uses only ≈8 of 22 possible misses in flight
2. **Laptop, all threads: bandwidth-bound**
   8.4 ms vs. 11.3 ms DRAM bound (L3 reuse) ⇒ no headroom
3. **Hábrók: neither**
   2.5–6.5× above both bounds: **barriers, imbalance, cross-socket traffic**

Seven optimisations, judged by IQR

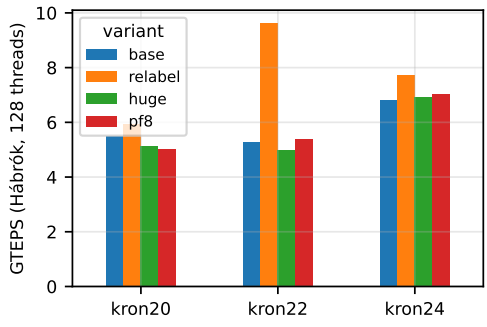


Significant: relabelling, thread count & placement, NUMA first touch

Within noise: compiler flags, schedules, huge pages, prefetch, α/β

O5: degree relabelling (up to 2.8×)

- **Why:** the model says bottom-up neighbour lines dominate Q
- Renumber by decreasing degree: hubs get small, adjacent IDs and come *first* in every sorted neighbour list
- Bottom-up hits a frontier vertex sooner: −35 % edges examined (kron20), −48 % (kron22)
- Laptop 8.58 → **3.53 ms** (2.4×); kron22: 2.8× laptop, 1.8× Hábrók
- Caveat: 0.46 s one-off cost, amortised after ≈90 searches



O2 + O6: fewer threads, better placement

O2: thread count and binding

- No OpenMP schedule beats upstream
- 48 threads bound to one socket: 2.85 → **1.38 ms (2.1×)**
- Smaller barriers and no cross-socket coherence

O6: NUMA-aware first touch

- `read()` puts all 125 MB of edges on *one* socket
- Parallel zeroing before the read spreads pages
- Spread threads: 4.00 → **1.94 ms (2.1×)**; kron22: −11 %
- No `numactl` on Hábrók ⇒ fixed in code

What did *not* help, and why

Strategy	Result	Explanation
O1 compiler flags	within IQR	no hot loop vectorises: early `break`, CAS *clobbers memory*; memory-bound anyway
O3 huge pages	within IQR	Hábrók already uses THP `always`; WSL2 variance hides it
O4 prefetch in top-down	$-11\,\%$ at best, n.s.	top-down is only 3 % of samples (Amdahl)
O7 α/β sweep	defaults near-optimal	$\alpha = 5$: stays top-down too long, $4.4\times$ slower

The analysis and the model predicted these outcomes *before* we implemented them.

Conclusions



- BFS is about **memory and synchronisation**, not arithmetic: IPC ≈ 1, 4.7 % of edges touched
- 1 core: latency-bound · laptop: bandwidth-bound · 128-core node: **barrier- and NUMA-bound**
- Validated toolchain: perf, PAPI, pecount, timers; gprof fails on OpenMP; verification exposed a data race
- Best gains: **relabelling (2.8×)**, **socket-local threads (2.1×)**, **NUMA first touch (2.1×)**
- Next: fuse a level's parallel regions to cut barriers; native-Linux laptop counters

Code, scripts, raw data: `github.com/CosminStamateAndrei/gapbs` (`pe/`)

Questions?

